Use Case BRD

Restaurant Table Reservation System

1. Restaurant Table Reservation System (end-to-end build)

Build a working web portal where customers reserve tables at a restaurant's branches, branch staff see the day's reservations, and an admin runs branches and tables. Every action must persist to a real database and survive an application restart.

2. Objective

The goal is to prove the ability to take a business need from requirement to a running, data-backed application, not just write isolated code. The restaurant reservation scenario touches every layer a production system has.

By the end, the build should demonstrate these skills:

Skill area	What we are checking
Requirement understanding	Reads this BRD, asks clarifying questions, and maps each requirement to a screen, API, and table
Data modelling	Designs normalized tables or collections with keys, relationships, and constraints
Backend engineering	Builds REST (or equivalent) APIs with validation, error handling, and correct status codes
Frontend engineering	Builds usable forms and lists that call the backend and show success and error states
Data integrity	Prevents double booking, tables too small for the party, invalid dates, and orphan records at the database level, not only in the UI
Security basics	Hashes passwords, enforces role-based access, and keeps secrets out of the code repository
Production readiness	Uses config files or environment variables, logs errors, and documents how to run the app
Deployment	Runs the app on a cloud host (preferred) or locally with clear setup steps
Communication	Explains design choices and trade-offs during the demo and Q&A;

3. Requirements and data flow

The portal has three roles and 16 functional requirements; the 12 marked Must are mandatory to pass. Should and Could items earn extra credit.

3.1 User roles

Role	How they get access	What they do
Customer	Self-registration	Finds branches, books, views, cancels, and modifies reservations
Staff	Account created by Admin, linked to one branch	Views the branch's reservations for today and upcoming days
Admin	Seeded in the database at setup	Manages branches and tables, monitors all reservations

3.2 Functional requirements

ID	Role	Requirement	Priority
FR-01	Customer	Register with name, email, phone, and password. Email must be unique.	Must
FR-02	All	Log in and log out; after login, land on the dashboard for that role.	Must
FR-03	Customer	Browse branches with address, opening hours, and seating areas (indoor, outdoor); filter by location.	Must
FR-04	Customer	Pick a branch, date, and party size to see available time slots only (future, within opening hours, a table that fits is free).	Must
FR-05	Customer	Book a slot with a special request; system assigns the smallest free table that fits and saves it with status BOOKED and a reference such as RES-20261027-0001.	Must
FR-06	System	Block double booking of the same table and slot, including two customers booking at the same moment.	Must
FR-07	Customer	View own reservations split into Upcoming and Past, with status.	Must
FR-08	Customer	Cancel a reservation; the table becomes free again.	Must
FR-09	Customer	Modify a reservation's time or party size to another available slot at the same branch.	Should
FR-10	Customer	Download a booked reservation as a PDF showing reference number, branch, date, time, party size, table, and status.	Should
FR-11	System	Show a reservation confirmation and record it in a notifications table.	Should
FR-12	Staff	View today's and upcoming reservations for their branch with customer name, party size, table, and special request.	Must
FR-13	Admin	Create, edit, and deactivate branches.	Must
FR-14	Admin	Create tables with branch, table number, seating area, and capacity; set each branch's opening hours and slot length.	Must
FR-15	Admin	View all reservations with filters for date, branch, and status.	Must
FR-16	Admin	Dashboard with counts of reservations per day, per branch, and cancellations.	Could

3.3 Business rules

• Slots are generated from each branch's opening hours and slot length (default 90 minutes).

• Customers can book from the next available slot up to 30 days ahead; past dates are rejected by the API, not only hidden in the UI.

• A table is assigned only if its capacity is equal to or greater than the party size; a party larger than every table is rejected with a clear message.

• A customer can hold only one active reservation per branch per day.

• Deactivated branches and tables disappear from availability, but their past reservations stay visible.

3.4 Non-functional requirements

Area	Requirement
Security	Passwords hashed (bcrypt or similar); every API checks role on the server; no secrets in the repository
Validation	Required fields, email and phone format, party size, and date rules checked on both frontend and backend
Error handling	Users see friendly messages; APIs return correct status codes (400, 401, 403, 404, 409); no stack traces on screen
Logging	Server logs each request and every error with a timestamp
Configuration	Database URL, port, and secrets read from environment variables or a config file

Area	Requirement
Setup	README with setup steps, a seed script (1 admin, 2 branches, 10 tables, 2 staff), and sample logins

4. Notes

4.1 Ground rules

• Any language, framework, and database is allowed. Low-code tools are not, since the aim is to see hands-on engineering.

• AI coding assistants may be used, but every part of the code must be explained in the walkthrough. Code that cannot be explained is scored as not done.

• Code lives in a Git repository with regular commits; a single final commit is a red flag.

• Hosting is a bonus. Free tiers (Render, Railway, Vercel with a managed database, or similar) are fine; no paid accounts are expected.

• Log questions about unclear requirements and the assumptions made; good assumptions are part of the score.

4.2 Deliverables

#	Deliverable
1	Git repository link with source code, README, seed script, and a .env.example file
2	Database schema (SQL script, migration files, or an ER diagram image)
3	API list (Postman collection, Swagger page, or a table in the README)
4	Short write-up (1 page): design choices, known gaps, and what they would improve next

Document end